HEALTHCARE DATA ENGINEERING • CASE STUDY

Clinical EHR Data Cleaning & Cascading Imputation

How I transformed 1,000 unstandardized Electronic Health Records into an audit-compliant, zero-null clinical dataset in Python & Pandas.

98.3%

COHORT PRESERVED

17 unrecoverable ghost records safely pruned

100%

NULL RESOLUTION

1,325 missing fields cleanly cleaned & imputed

HIPAA

AUDIT PROVENANCE

100% auditable via explicit Yes/No flags

Daniel .A. Dongo

Data Analyst & Engineer • Python • Pandas • Healthcare Analytics

SWIPE TO EXPLORE ➡️

01 • PROBLEM STATEMENT

The Challenge: Real-World EHR Data is Messy

Why naive data cleaning breaks downstream medical machine learning.

1. Composite Vitals

Formatting

- Blood pressure stored as text string (e.g. `"120/80"`).
- Whitespace corruption prevents numeric analysis.
- Required vectorized decomposition into systolic and diastolic.

2. Corrupted Types

Data Types

- The age column contained verbal words (`"forty"`).
- Decimal floats on completed integer patient years.
- Required strict type coercion and nullable `Int64` casting.

3. Polymorphic Dates

Parsing

- Dates recorded with inconsistent syntax across visits.
- Mixed delimiters (`"2023_04_15"` vs `"15-Apr-2023"`).
- Standardized 100% of dates using fuzzy token parsing.

4. The "Ghost Patient" Risk

Clinical Risk

- Blind imputation on empty records fabricates synthetic patients.
- Naive listwise deletion drops nearly 10% of real patients.
- Needed an objective clinical triage rule to preserve power.

CORE CLINICAL DILEMMA:

How do we resolve 1,325 missing values across 1,000 records without inventing synthetic patients or destroying statistical power?

Solution: Domain-driven triage filter + stratified cascading median imputation.

02 • PIPELINE ARCHITECTURE

End-to-End Pipeline Workflow

A 5-stage production framework engineered for clinical reproducibility.

01

Text Normalization & Vital Splitting

Decomposed composite `blood_pressure` into numeric `systolic_bp` & `diastolic_bp`. Fixed typos (`"forty"` ➔ 40) and stripped all whitespace.

02

Resilient Timestamp Standardization

Applied `dateutil.parser` with `fuzzy=True` to resolve polymorphic delimiters, converting 100% of dates to `datetime64[ns]` without data loss.

03

The "Ghost Patient" Triage Filter

Audited 4 core clinical vitals (age, blood pressure, condition, cholesterol). Pruned 17 unrecoverable records (1.70% data loss) to prevent synthetic profile hallucination.

04

Hierarchical Cascading Median Imputation

3-tier stratified fallback: `(Age Group + Gender)` ➔ `Gender Median` ➔ `Global Median`. Preserves biological variance across demographics.

05

Regulatory Audit Flags & Anomaly Review

Created explicit provenance flags (`*_imputed`) mapped to `'Yes'/'No'` for FDA/HIPAA compliance. Audited physiological bounds and IQR outliers.

03 • CODE IMPLEMENTATION

Snapshot: Vital Decomposition & Triage

Vectorized Pandas operations for feature extraction and row filtering.

1. Vectorized Blood Pressure Splitting (Cells 21–24)

```
# 1. Split composite slash-delimited blood pressure string df[['systolic_bp', 'diastolic_bp']] =
df['blood_pressure'].str.split('/', expand=True) # 2. Strip whitespace and safely coerce non-numeric entries
df['systolic_bp'] = pd.to_numeric(df['systolic_bp'].str.strip(), errors='coerce') df['diastolic_bp'] =
pd.to_numeric(df['diastolic_bp'].str.strip(), errors='coerce') # 3. Cast to nullable integer to preserve clean schema
df[['systolic_bp', 'diastolic_bp']] = df[['systolic_bp', 'diastolic_bp']].round().astype('Int64')
```

2. Clinical Triage Filter: Pruning Ghost Patients (Cells 27–32)

```
# Define core clinical diagnostic markers critical_columns = ['age', 'blood_pressure', 'condition', 'cholesterol'] #
Count missing critical values per patient missing_per_row = df[critical_columns].isna().sum(axis=1) # Prune patients
missing >=3 core markers (17 rows / 1.70%) # Avoids 'inventing patients' while preserving 98.3% of the cohort df =
df[missing_per_row < 3].copy()
```

04 • CODE IMPLEMENTATION

Snapshot: Cascading Imputation

Stratified median hierarchy preserving demographic variation.

3. Hierarchical Cascading Median Imputation & Audit Tracking (Cells 39–50)

```
imputed_cols = ['systolic_bp', 'diastolic_bp', 'cholesterol'] for col in imputed_cols: df[col] = pd.to_numeric(df[col], errors='coerce') # Tier 1: Demographic subgroup median (Age Bracket + Biological Sex) subgroup_med = df.groupby(['age_group', 'gender'], observed=False)[col].transform('median') # Tier 2: Biological sex fallback median gender_med = df.groupby('gender', observed=False)[col].transform('median') # Tier 3: Global population median global_med = df[col].median() # Apply cascade fallback df[col] = df[col].fillna(subgroup_med).fillna(gender_med).fillna(global_med) # Convert boolean audit flags to regulatory 'Yes' / 'No' string labels bool_cols = df.select_dtypes(include='bool').columns df[bool_cols] = df[bool_cols].replace({True: 'Yes', False: 'No'})
```

WHY HIERARCHICAL MEDIANS OVER GLOBAL MEANS?

A 25-year-old female and a 75-year-old male have completely different healthy vital baselines.
Stratifying by age bracket and biological sex preserves genuine clinical distributions.

05 • BUSINESS & CLINICAL IMPACT

Results: Before vs. After

100% missing data resolution with verified clinical integrity.

1,000 → 983

PATIENTS RETAINED

98.3% of the patient cohort preserved; 17 unrecoverable ghost profiles pruned.

1,325 → 0

MISSING VALUES

100% missing values resolved across all 15 clinical and demographic columns.

0% → 100%

VITALS USABILITY

Blood pressure decomposed into numeric systolic & diastolic columns for risk scoring.

100% Auditable

REGULATORY LINEAGE

All imputed cells tracked via explicit Yes/No flags adhering to HIPAA/GAMP standards.

EXPORTED DELIVERABLE: cleaned_healthcare_data.csv (983 Rows × 15 Columns)

- ✓ **Production-Grade Schema:** Zero nulls remaining, validated data types, normalized categorical labels.
- ✓ **Predictive Analytics Ready:** Clinical vitals, demographic brackets, and time-series visit dates ready for ML models.
- ✓ **Auditable Lineage:** Transparent provenance tracking ensuring compliance with FDA 21 CFR Part 11.

06 • SUMMARY & OPEN SOURCE

Key Takeaways & Repository

Core engineering lessons and link to the open-source repository.

1. Clinical Domain Rules > Generic Data Cleaning

Lesson 1

- Healthcare data cannot be treated like generic e-commerce clickstream data.
- Imputing everything blindly invents patients; naive listwise deletion destroys real patients.
- Objective triage filters based on core vitals strike the optimal statistical balance.

2. Explainability Matters in Regulated Environments

Lesson 2

- Stratified cascading medians are fast, 100% explainable, and have zero model-drift overhead.
- Explicit boolean audit flags (`*_imputed`) ensure compliance with HIPAA and FDA 21 CFR.
- Essential for medical AI models where training data provenance is heavily audited.

EXPLORE THE FULL OPEN-SOURCE PROJECT:

github.com/ELSHINE123/healthcare-data-cleaning-pipeline

Developed by: **Daniel .A. Dongo** • linkedin.com/in/daniel-dongo

VIEW REPO ⭐